



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Lossless coding principles

Marco Cagnazzo

Multimedia Communications



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques



Outline

Introduction

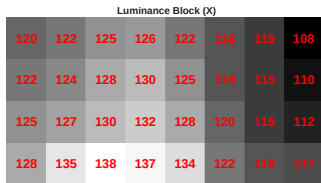
Principles of Information Theory

Optimal coding

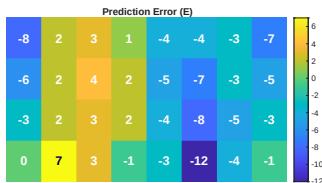
Other Techniques



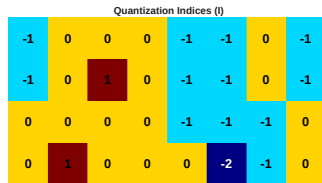
From Samples to Bits: The Encoding Process



(A) Original Luminance $x(n)$



(B) Prediction Error $y(n)$



(C) Quantization Indices $i(n)$

1. **Spatial Prediction:** Leverages local correlation to reduce variance. $y(n) = x(n) - \hat{x}(n)$
 2. **Quantization:** Maps the high-precision residuals into a finite set of discrete indices $i(n)$.
- The Central Question: How do we map the resulting indices into an efficient bitstream?

The Answer: **Lossless Coding**

The chosen code must satisfy two fundamental requirements: **Uniquely Decodable:** one-to-one mapping from the bitstream back to the symbol **sequence**

Compact: smallest possible average codeword length



Fixed-Length Coding (FLC): The Baseline

FLC Mapping Table (Two's complement)

Index	4-bit Code	Index	4-bit Code
-7	1001	1	0001
-6	1010	2	0010
-5	1011	3	0011
-4	1100	4	0100
-3	1101	5	0101
-2	1110	6	0110
-1	1111	7	0111
0	0000	(spare)	1000

Straightforward parsing: the decoder segments the bitstream every $L = 4$ bits

Robustness: a single bit error only affects one symbol

Rate: it is fixed, $R = \log_2 L$

Can we do better?

- ▶ FLC neglects that indices might be not **equally probable**
- ▶ This happens e.g. with predictive quantization
- ▶ *Idea:* Assign shorter bitstrings to probable symbols (like 0) and longer ones to rare symbols (like -7 or 7).



Variable-Length Coding (VLC)

If we use codewords of different lengths, we face a major hurdle: **The Parsing Problem** How does the decoder "slice" the continuous bitstream into individual symbols if it doesn't know where one ends and the next begins?

Key Questions:

1. **The Theoretical Bound:** How much can we actually gain by using VLC? *Answer:* The limit is defined by the **Source Entropy** $H(X)$.
2. **Practical Implementation:** How do we achieve this limit while ensuring unique decodability? *Answer:* **Huffman** coding achieves the best theoretical performance, but faces scalability problems. **Arithmetic coding** is sub-optimal, but asymptotically optimal and scales easily with the input size
3. All these approaches assume that we know data statistics: what if we don't? *Answer:* Universal coding (LZW)



Terminology of lossless coding

Lossless compression is based on data statistical properties. The basic idea is very simple: instead of using fixed-length coding, we use variable-length coding (VLC) with

- ▶ **Short** codewords for **probable** symbols
- ▶ **Long** codewords for **not probable** symbols

Definitions:

Alphabet : $\mathcal{X} = \{x_1, x_2, \dots, x_M\}$ set of symbols to encode

- ▶ $\{0, 1, \dots, 255\}$ for luminance values
- ▶ Latin alphabet for a text in French, English, ...

Code : application between $\mathcal{X}$ and $\{0, 1\}^*$, the set of finite-length bit strings

- ▶ Fixed-length coding
- ▶ Variable-length coding



Choosing a code

Code: $\mathcal{C} : x_i \in \mathcal{X} \rightarrow c_i \in \{0, 1\}^*$

Fixed-length coding (FLC)

- ▶ All codewords have the same length
- ▶ E.g., if $M = 256$ symbols, we need $\lceil \log M \rceil = 8$ bits to encode each symbol
- ▶ For a text, $M = 26$, we need $\lceil \log M \rceil = 5$ bpS (bit per symbol)

Variable-length coding (VLC)

- ▶ ℓ_i : length of codeword c_i
- ▶ We achieve *lossless coding* if:
 - ▶ Decodability condition: prefix condition
 - ▶ Non-equiprobable symbols



Example: text compression

Method	FLC
Number of symbols M	26
Coding rate ($\mathcal{L}$)	5 bpS
Compression ratio	1

- ▶ Each letter is represented on 5 bits
- ▶ No compression (this is our reference)



Examples of codes

Symbol	Code 1	Code 2	Code 3	Code 4
A	0	0	0	0
B	11	10	01	01
C	111	110	011	11

- ▶ Code 1 is not uniquely decodable : 111111 can be decoded as BBB or CC
- ▶ Code 2 is instantaneously decodable (**prefix condition**)
- ▶ Code 3 is decodable with a delay less than or equal to 1 bit
- ▶ Code 4 is decodable but with undefined delay: 0111111...11 can be decoded as ACCC...CC or BCC...C depending on the number of 1's is even or odd



Examples of codes

Symbol	Prob	Code 1	Code 2	Code 3	Code 4
A	1/2	0	0	0	0
B	1/4	0	1	10	01
C	1/8	1	00	110	011
D	1/8	10	11	111	0111

What is the best code? Go to wooclap.com with the code **GYKKIN**



Examples of codes

Symbol	Prob	Code 1	Code 2	Code 3	Code 4
A	1/2	0	0	0	0
B	1/4	0	1	10	01
C	1/8	1	00	110	011
D	1/8	10	11	111	0111

What is the best code? Go to wooclap.com with the code **GYKKIN**

- ▶ Code 1 is not uniquely decodable A and B have the same codeword and moreover 10 can be decoded in three different ways: CA, CB, D
- ▶ Code 2 is not uniquely decodable since 00 can be AA or C and 11 can be BB or D
- ▶ Code 3 is instantaneously decodable (prefix condition)
- ▶ Code 4 is decodable but with a delay of one bit



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques



McMillan's theorem and Kraft's inequality

- ▶ **McMillan's theorem:** Decodable codes do not improve performance with respect to instantaneous codes

We can focus only on instantaneous codes

- ▶ **Kraft's inequality:** there exists an instantaneous code with lengths $\{\ell_1, \dots, \ell_M\}$ **if and only if:**

$$\sum_i 2^{-\ell_i} \leq 1$$

We have a mathematical characterization of instantaneous codes

- ▶ It is a **necessary and sufficient** condition for the existence of a prefix code.
- ▶ If the sum equals 1, the code is said to be **complete** (or maximal).
- ▶ It sets the stage for Shannon's Source Coding Theorem.



Proof of Necessity

Prefix code

A: 00

B: 010

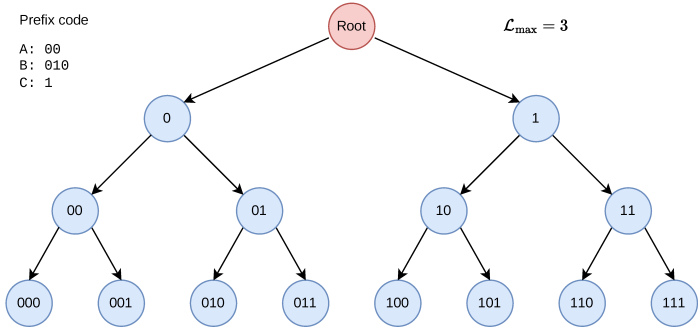
C: 1

$$\mathcal{L}_{\max} = 3$$

Given a prefix code, build a binary tree of depth $L_{\max} = \max(\ell_i)$



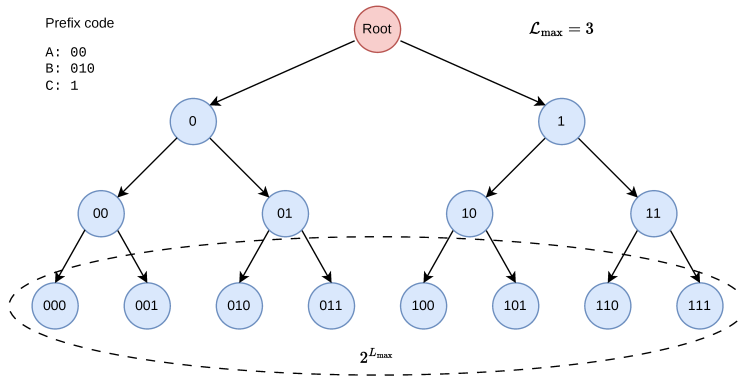
Proof of Necessity



Given a prefix code, build a binary tree of depth $L_{max} = \max(\ell_i)$



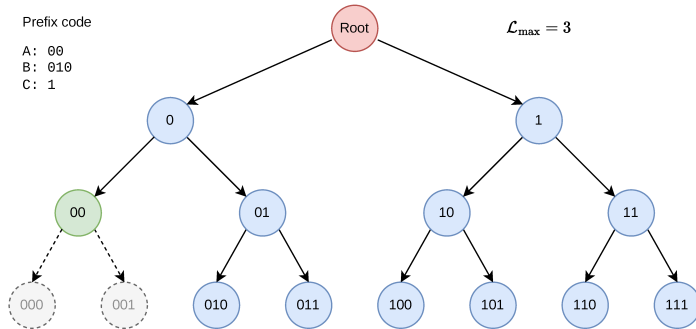
Proof of Necessity



The total number of leaf nodes at level $L_{\max}$ is $2^{L_{\max}}$



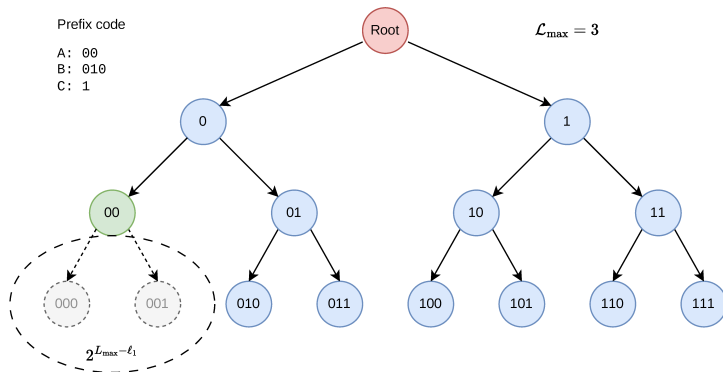
Proof of Necessity



Each codeword c_i of length ℓ_i "occupies" a subtree



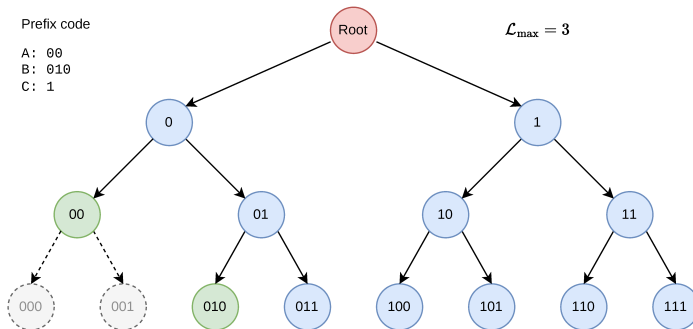
Proof of Necessity



The number of descendant leaves at level $L_{\max}$ "blocked" by c_i is: $\text{Leaves}(c_i) = 2^{L_{\max} - \ell_i}$



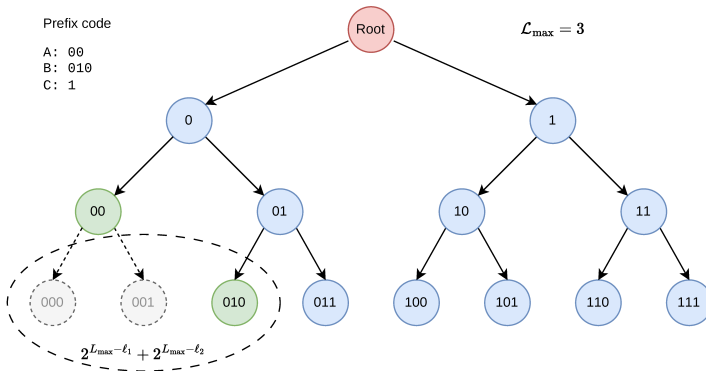
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



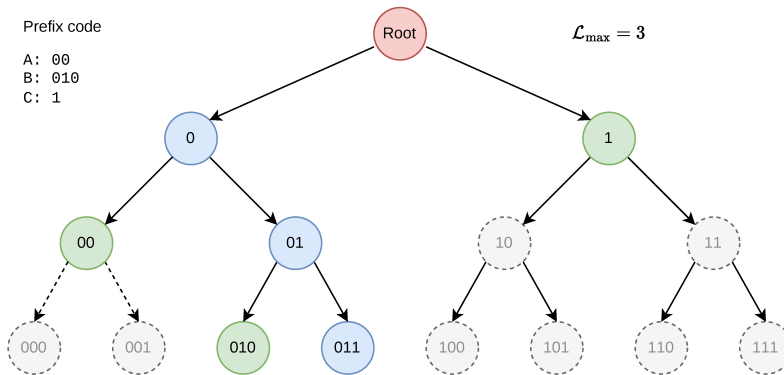
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



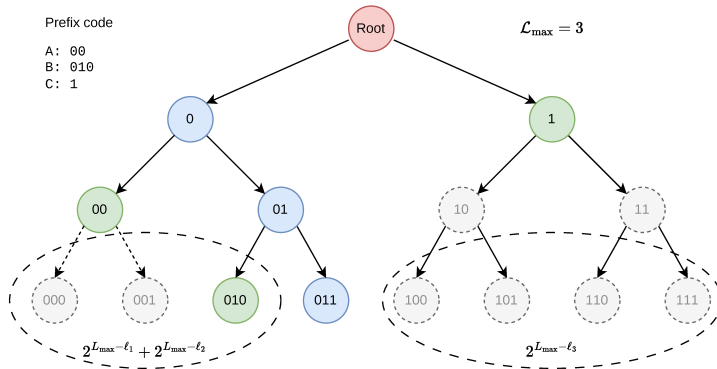
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



Proof of Necessity



The total number of blocked leaves cannot exceed the available leaves: $\sum_{i=1}^N 2^{L_{\max}-\ell_i} \leq 2^{L_{\max}}$

Dividing by $2^{L_{\max}}$ yields the Kraft inequality: $\sum_{i=1}^N 2^{-\ell_i} \leq 1$



Proof of Sufficiency

Goal: Given $\{\ell_i\}$ such that $\sum 2^{-\ell_i} \leq 1$, construct a prefix code.

Greedy Construction Algorithm:

1. Sort lengths: $\ell_1 \leq \ell_2 \leq \dots \leq \ell_N$.
2. At each step k , pick an available node at depth ℓ_k .
3. Mark all its descendants as "forbidden".



Proof of Sufficiency

Goal: Given $\{\ell_i\}$ such that $\sum 2^{-\ell_i} \leq 1$, construct a prefix code.

Greedy Construction Algorithm:

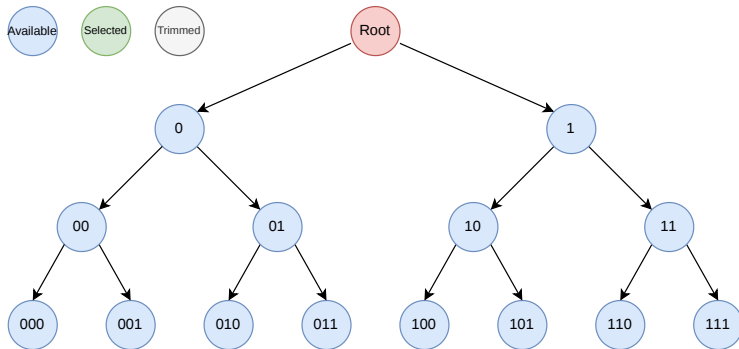
1. Sort lengths: $\ell_1 \leq \ell_2 \leq \dots \leq \ell_N$.
2. At each step k , pick an available node at depth ℓ_k .
3. Mark all its descendants as "forbidden".

Existence Argument: At step k , the "consumed capacity" of the tree is $\sum_{i=1}^{k-1} 2^{-\ell_i}$. Since the total sum is ≤ 1 , and $\ell_k \geq \ell_{k-1}$, there must be at least one branch available at depth ℓ_k to place the next codeword.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$

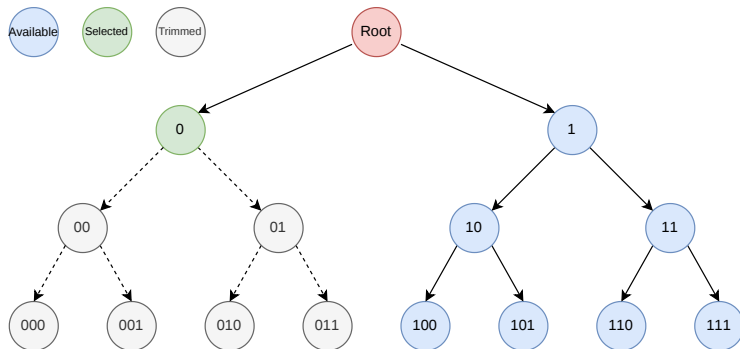


Build a complete binary tree, depth equal to $L_{\max}$



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



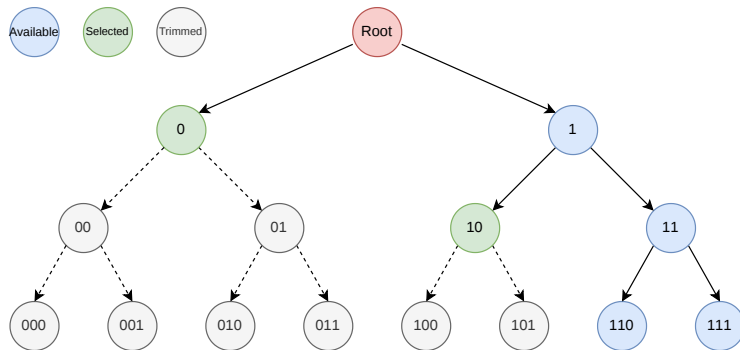
Select a node at depth ℓ_1 , trim its subtree

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



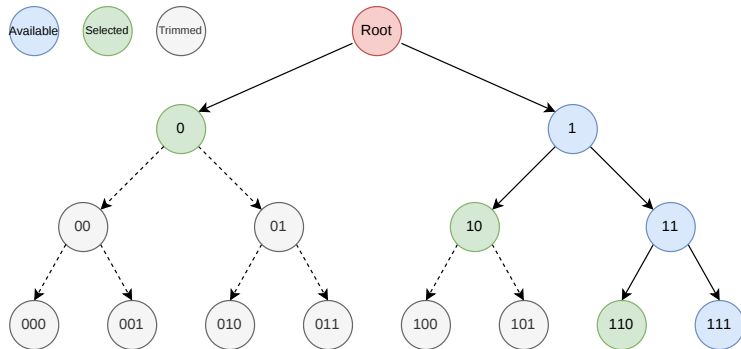
Select an available node at depth ℓ_2 , trim its subtree

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



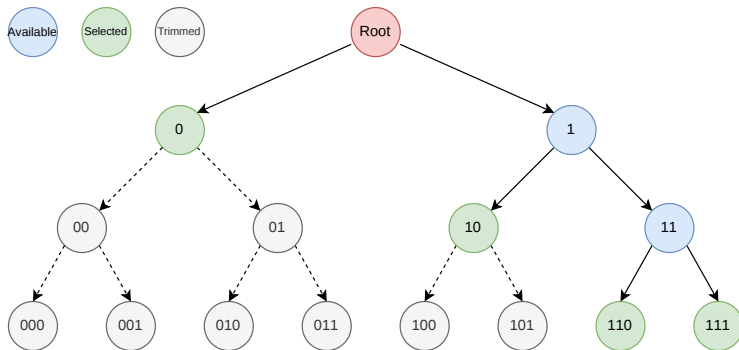
Select an available leaf

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



Select an available leaf.
The code is $\{0, 10, 110, 111\}$

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Information and entropy

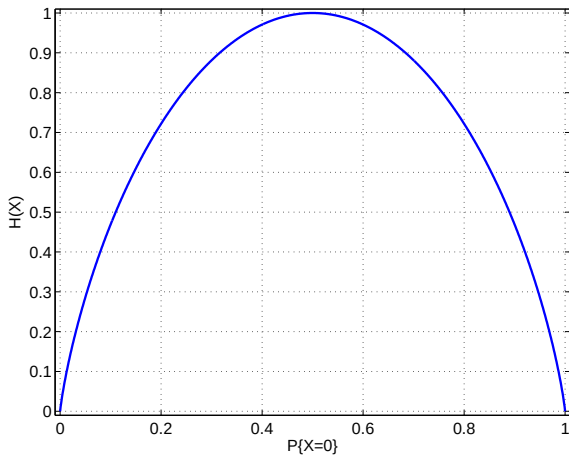
- ▶ RV model of the information source: X
- ▶ Symbol x_i has probability p_i
- ▶ Information associated to x_i is $I(x_i) = -\log p_i$
 - ▶ $I(x_i) \geq 0$
 - ▶ If $p_i = 1$, $I = 0$
 - ▶ If two symbols are independent, $I(x_i, x_j) = I(x_i) + I(x_j)$
- ▶ Source entropy: average symbol information

$$H(X) = -\sum_i p_i \log p_i$$

- ▶ Interpretation:
 - ▶ Average uncertainty associated to the random variable X



Binary variable entropy



Case of binary RV

Alphabet: $\{0, 1\}$

The probability mass function (pmf) is given by:

$$p = P\{X = 0\}$$

$$q = P\{X = 1\} = 1 - p \quad \text{as a consequence}$$

$$H(X) = -p \log(p) - (1 - p) \log(1 - p)$$

The entropy can be traced as a function of a single parameter p



Distribution with maximum entropy

Constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \quad \sum_{i=1}^M p_i = 1$$

In order to solve this **constrained optimization problem**, we need a specific tool: the Lagrange's method (or method of Lagrange's multipliers)



Constrained optimization: Lagrange's method

Let's consider a function $f : x \in \mathbb{R}^n \rightarrow \mathbb{R}$

In order to find the maximum or minimum of a function $f(x)$ subjected to the equality constraint $\phi(x) = 0$, we look for the stationary points of the functional

$$J(x, \lambda) = f(x) + \lambda \cdot \phi(x)$$

with $\lambda \in \mathbb{R}$. The solution is among these points.

The stationary points are computed by setting to zero all the derivatives of J :

$$\frac{\partial J}{\partial x_i} = 0 \quad \frac{\partial J}{\partial \lambda} = 0$$



Distribution with maximum entropy

Let us go back to our constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \qquad \sum_{i=1}^M p_i = 1$$

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method:



Distribution with maximum entropy

Let us go back to our constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \qquad \sum_{i=1}^M p_i = 1$$

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method: $p^* = [p_1^* \ p_2^* \ \dots \ p_M^*]$ such that $p_i^* = \frac{1}{M} \quad \forall i \in \{1, 2, \dots, M\}$

$$J(p) = - \sum_{i=1}^M p_i \log p_i + \lambda \left(\sum_{i=1}^M p_i - 1 \right) \qquad \frac{\partial J}{\partial p_i}(p^*) = 0$$

$$\frac{\partial J}{\partial p_i} = - \left(\frac{\log e}{p_i} p_i + \log p_i \right) + \lambda \qquad \log p_i^* = \lambda - \log e = \text{constant}$$



Joint entropy

- ▶ For a couple of r.v.'s X and Y
- ▶ Joint probability distribution $p_{i,j} = P\{X = x_i, Y = y_j\}$
- ▶ Joint entropy: average information of couples

$$H(X, Y) = - \sum_{i,j} p_{i,j} \log p_{i,j}$$

- ▶ Formally, there is no difference between the joint probability of a couple X, Y and the entropy of a single r.v. having the same set of probabilities
- ▶ The entropy does not depends on the values, only on the probabilities



Conditional entropy

- ▶ We consider a couple of r.v. X and Y
- ▶ Let $p_j = P\{Y = y_j\}$
- ▶ Conditional entropy:

$$H(X|Y) = \sum_j p_j H(X|Y = y_j)$$

- ▶ It is easy to show that:

$$\begin{aligned} H(X, Y) &= H(Y) + H(X|Y) \\ &= H(X) + H(Y|X) \end{aligned}$$



Properties

$$H(X) \geq 0$$

$$H(X, Y) = H(Y) + H(X|Y)$$

$$H(X) + H(Y|X)$$

$$H(X, Y) \leq H(X) + H(Y)$$

$$H(X|Y) \leq H(X)$$

$$H(X) \leq \log_2 M$$

with equality $\Leftrightarrow$ independence

with equality $\Leftrightarrow$ independence

with equality $\Leftrightarrow X \sim \mathcal{U}$



Conditional Entropy

Formal Definition

- ▶ The average uncertainty about X given Y : $H(X|Y) = \sum_j p_j H(X|Y = y_j)$
- ▶ **The Chain Rule:**
 $H(X, Y) = H(Y) + H(X|Y)$

What does it represent?

- ▶ **Residual Uncertainty:** The amount of information still required to describe X after Y has been observed.
- ▶ **Equivocation:** In communications, it is the information lost due to a noisy channel.

If X and Y are independent, $H(X|Y) = H(X)$.
If X is a deterministic function of Y , $H(X|Y) = 0$.



Properties

Fundamental Property: Conditioning Reduces Entropy

$$H(X|Y) \leq H(X)$$

"Information never hurts": knowing Y can only decrease (or leave unchanged) the uncertainty about X .



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:
 - ▶ $\forall i = 1, \dots, M, \ell_i \in \mathbb{N}$



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:
 - ▶ $\forall i = 1, \dots, M, \ell_i \in \mathbb{N}$
 - ▶ The Kraft's inequality is satisfied (prefix code)



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:
 - ▶ $\forall i = 1, \dots, M, \ell_i \in \mathbb{N}$
 - ▶ The Kraft's inequality is satisfied (prefix code)
 - ▶ The average length: $\mathcal{L} = \sum_{i=1}^M p_i \ell_i$ is minimized among all possible sets of ℓ_i 's
- ▶ We formulate the problem using the Lagrangian multipliers



Optimal code

- ▶ But we need to drop condition $\ell_i \in \mathbb{N}$
- ▶ Constrained minimization:

$$\underline{\ell}^* = \arg \min_{\underline{\ell}} \sum_i p_i \ell_i$$

$$\text{subject to } \sum_i 2^{-\ell_i} = 1$$



Optimal code

- ▶ But we need to drop condition $\ell_i \in \mathbb{N}$
- ▶ Constrained minimization:

$$\underline{\ell}^* = \arg \min_{\underline{\ell}} \sum_i p_i \ell_i$$

$$\text{subject to } \sum_i 2^{-\ell_i} = 1$$

$$J(\underline{\ell}) = \sum_i p_i \ell_i + \lambda \left(\sum_i 2^{-\ell_i} - 1 \right)$$

$$\frac{\partial J}{\partial \ell_i} = p_i - (\lambda \ln 2) 2^{-\ell_i^*} = 0$$

$$\sum_i p_i = (\lambda \ln 2) \sum_i 2^{-\ell_i^*}$$

$$1 = \lambda \ln 2$$

$$2^{-\ell_i^*} = p_i$$

$$\ell_i^* = -\log_2 p_i$$

$$\mathcal{L}^* = \sum_i -p_i \log_2 p_i = H(X)$$



Why the Equality in Kraft's Constraint?

To minimize $L = \sum p_i \ell_i$, we must use the available "prefix space" efficiently.

The "Unused Branch" Argument:

- ▶ If $\sum 2^{-\ell_i} < 1$, there exists at least one leaf in the binary tree that is neither a codeword nor a descendant of one.
- ▶ We could **shorten** some codewords by moving them "up" the tree.
- ▶ Shortening ℓ_i strictly decreases L (since $p_i > 0$).

Conclusion: The minimum average length L^* must satisfy the **active constraint**:

$$\sum 2^{-\ell_i^*} = 1$$



Shannon's source coding theorem

The minimum average length $\mathcal{L}^* = H(X)$ can be achieved if and only if the lengths are *dyadic*:

$$\forall i \in \{1, 2, \dots, M\}, \exists k \in \mathbb{N} \mid p_i = 2^{-k}$$

Therefore, in the general case, the average length can only be greater than or equal to the entropy:

$$\mathcal{L}^* \geq H(X)$$

with equality if and only if the lengths are dyadic

This is the **Shannon's Source Coding Theorem**



Entropy coding

Shannon's theorem:

Optimal coding rate $\geq$ source entropy

therefore optimal coding is called *Entropy coding*.

- ▶ It is a strict identity if the probability distribution is *dyadic*, i.e.,
 $\forall i, \exists k \in \mathbb{N} : p_i = 2^{-k}$
- ▶ However, when M increases, the optimal coding rate is practically very close to H



Entropy coding

- ▶ It easy to find an instantaneous code such that $\mathcal{L} < H(X) + 1$, we just take $\ell_i = \lceil -\log_2 p_i \rceil$
- ▶ Kraft's inequality is satisfied:

$$\ell_i = -\log p_i + \delta_i$$

$$0 \leq \delta_i < 1$$

$$2^{-\ell_i} = p_i \times 2^{-\delta_i} = \epsilon_i p_i$$

$$\frac{1}{2} < \epsilon_i \leq 1$$

$$\sum_i 2^{-\ell_i} = \sum_i \epsilon_i p_i \leq \sum_i p_i = 1$$

- ▶ The average length is obtained as follows:

$$\ell_i = \lceil -\log_2 p_i \rceil < -\log_2 p_i + 1$$

$$p_i \ell_i < -p_i \log_2 p_i + p_i$$

$$\sum_i p_i \ell_i < \sum_i (-p_i \log_2 p_i + p_i)$$

$$\mathcal{L} < H(X) + 1$$

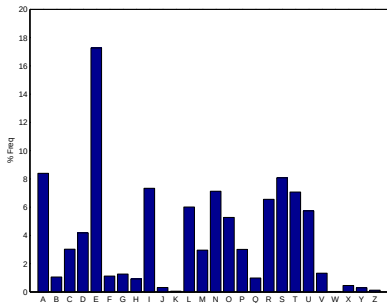
- ▶ Since $\mathcal{L}^* \leq \mathcal{L}$,

$$H(X) \leq \mathcal{L}^* < H(X) + 1$$



Example: French text compression

Source entropy	3.999 bpS
Method	Any VLC
Coding rate ($\mathcal{L}$)	≥ 3.999 bpS
Compression ratio	≤ 1.25



Letter distribution in a French text



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques



Huffman coding

Huffman discovered how to build the optimal lossless coder for any source with known probabilities. **Steps:**

1. Create leaf nodes for each symbol, weighted by p_i
2. While there's more than one node:
 - ▶ Select two nodes with lowest weights
 - ▶ Create a new internal node as their parent
 - ▶ Set new node's weight as sum of children's weights
3. Assign '0' to left edges, '1' to right edges
4. Read code for each symbol from root to leaf



Huffman coding

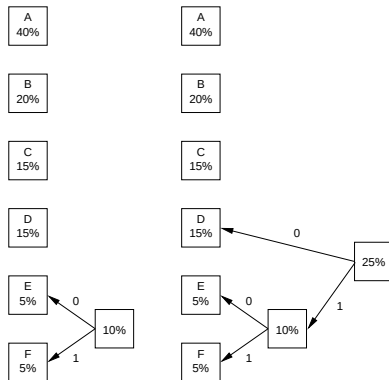
Example:

Symbol	Probability
A	0.4
B	0.2
C	0.15
D	0.15
E	0.05
F	0.05

6 symbols, 3 bits per symbol (without coding)

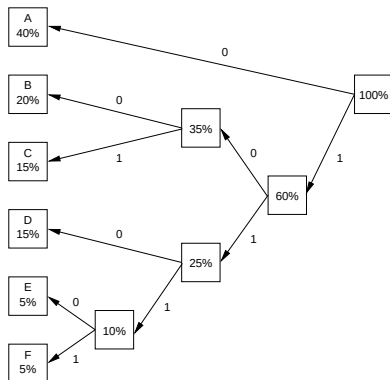


Huffman coding





Huffman coding





Huffman coding

Symbol	Probability	Codeword
A	0.4	0
B	0.2	100
C	0.15	101
D	0.15	110
E	0.05	1110
F	0.05	1111

Find $\mathcal{L}$ and H



Huffman coding

Symbol	Probability	Codeword
A	0.4	0
B	0.2	100
C	0.15	101
D	0.15	110
E	0.05	1110
F	0.05	1111

Find $\mathcal{L}$ and H

$$\mathcal{L} = 0.4 \cdot 1 + 0.2 \cdot 3 + 0.15 \cdot 3 + 0.15 \cdot 3 + 0.05 \cdot 4 + 0.05 \cdot 4 = 2.3 \text{ bits/Symbol}$$

$$H = 0.4 \cdot \log_2 \frac{1}{0.4} + 0.2 \cdot \log_2 \frac{1}{0.2} + 2 \cdot 0.15 \cdot \log_2 \frac{1}{0.15} + 2 \cdot 0.05 \cdot \log_2 \frac{1}{0.05}$$

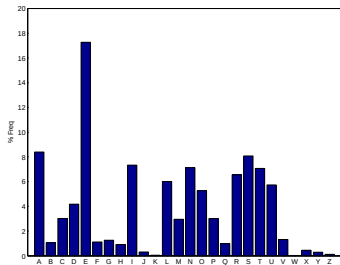
$$\cong 2.2464 \text{ bits/Symbol}$$

Go to wooclap.com with the code **GYKKIN**

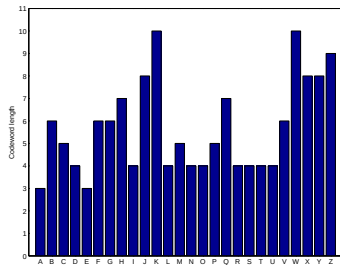


Example: Compression of French text

Method	Huffman
Source entropy	3.999 bpS
Coding rate ($\mathcal{L}$)	4.041 bpS
Compression ratio	1.238



Letter probabilities in a French text



Huffman (i.e., optimal) codeword lengths



Huffman coding

- ▶ Example: coding a scanned document. Typically many pixels are white and only few are black. If X is a r.v. representing the color of a pixel:
- ▶ $\Pr(\{X = B\}) = p \ll 1$
- ▶ $\Pr(\{X = W\}) = 1 - p$
- ▶ $H(X) \ll 1$
- ▶ Huffman coding: $B \rightarrow 0 \quad W \rightarrow 1$
- ▶ $\mathcal{L} = 1 \gg H(X)$
- ▶ Conclusion: the entropy is very small, but the optimal code has a coding rate much larger than the entropy: can we do better?



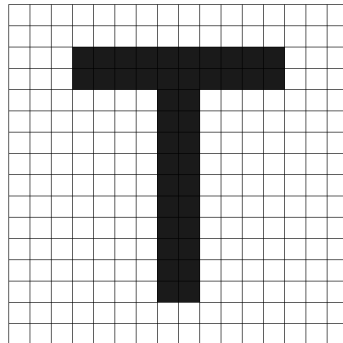
Huffman coding

$$H(X) = 0.586$$

Huffman code:

■	0
□	1

Code length: 1 bpp

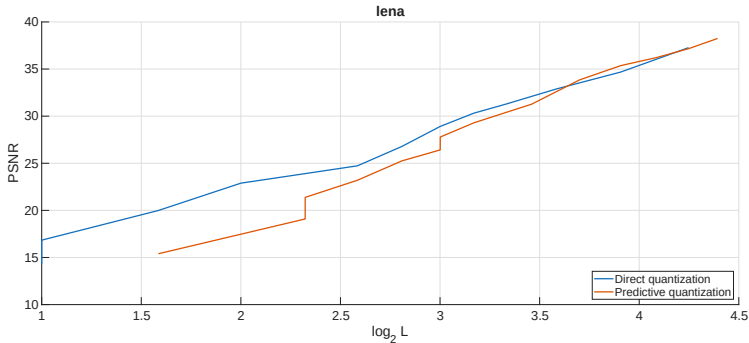


$$P(\square) = 86.7\%$$

$$P(\blacksquare) = 13.3\%$$



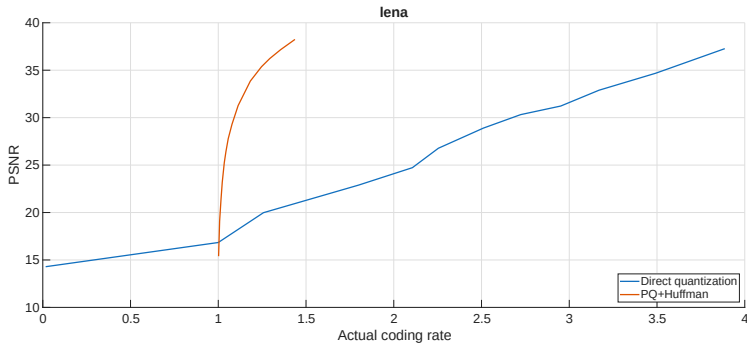
Huffman Coding and Predictive Quantization



Quantized prediction errors encoded with **FLC** (Fixed-Length)



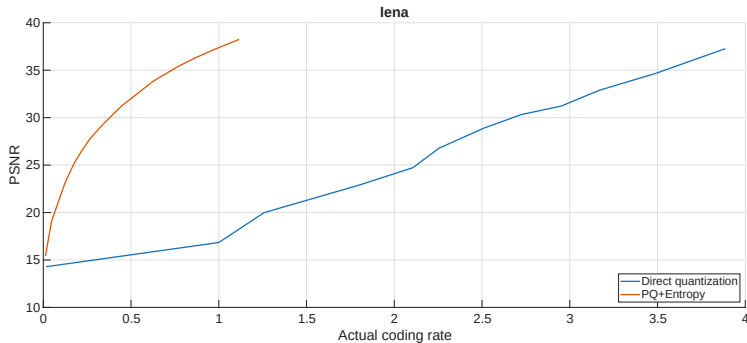
Huffman Coding and Predictive Quantization



Quantized prediction errors encoded with **Huffman** (VLC)



Huffman Coding and Predictive Quantization



Theoretical limit: **Entropy** of quantized prediction errors



Huffman coding

How to improve performance?

- ▶ Let us refer to the block of K symbols $X^K = (X_1 X_2 \dots X_K)$:

$$H[(X_1 X_2 \dots X_K)] \leq \sum_{i=1}^K H(X_i)$$

with equality if and only if components of X^K are **independent** each from the other

- ▶ Idea of **block coding**: Encode the block X^K rather than each individual component
 1. If the symbols are not independent, $H(X^K)/K < H(X_i)$
 2. $\mathcal{L} < H(X^K) + 1 \Leftrightarrow \mathcal{L}_S < H(X^K)/K + 1/K$

If the symbols are identically distributed (but not necessarily independent),
 $\mathcal{L}_S < H(X^K)/K + 1/K \leq H(X_i) + 1/K$



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels
- ▶ $\Pr(BB) = p^2$
- ▶ $\Pr(WB) = p(1 - p) \approx p$
- ▶ $\Pr(BW) = p(1 - p) \approx p$
- ▶ $\Pr(WW) = (1 - p)^2 \approx 1 - 2p$
- ▶ $H(X_1, X_2) = 2H(X) \ll 2$
- ▶ Huffman coding:



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels
- ▶ $\Pr(BB) = p^2$
- ▶ $\Pr(WB) = p(1 - p) \approx p$
- ▶ $\Pr(BW) = p(1 - p) \approx p$
- ▶ $\Pr(WW) = (1 - p)^2 \approx 1 - 2p$
- ▶ $H(X_1, X_2) = 2H(X) \ll 2$
- ▶ Huffman coding: $WW \rightarrow 0$ $WB \rightarrow 10$ $BW \rightarrow 110$ $BB \rightarrow 111$
- ▶ $\mathcal{L} \approx 1$
- ▶ $\mathcal{L}_S \approx \frac{1}{2}$



Huffman coding

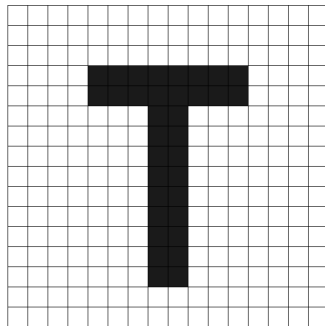
$$H([X_1 X_2]) = 1.022$$

$$H([X_1 X_2])/2 = 0.511$$

Huffman code:

■ ■	00
□ ■	010
■ □	011
□ □	1

Code length: 1.3 bits per block, **0.65 bpp**



$$P(\square\square) = 80.0\%$$

$$P(\square\square) = 5.00\%$$

$$P(\blacksquare\square) = 5.00\%$$

$$P(\blacksquare\square) = 10.0\%$$

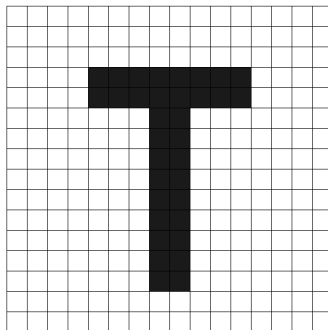


Huffman coding

$$H([X_1 X_2 X_3 X_4]) = 1.533$$

$$H([X_1 X_2 X_3 X_4])/4 = 0.383$$

Code length: 1.733 bits
per block, **0.433** bpp



$P(\begin{smallmatrix} \square & \square \\ \square & \square \end{smallmatrix}) = 74.7\%$	$P(\blacksquare) = 7.56\%$
$P(\begin{smallmatrix} \square & \square \\ \blacksquare & \square \end{smallmatrix}) = 0.89\%$	$P(\begin{smallmatrix} \square & \square \\ \square & \blacksquare \end{smallmatrix}) = 0.89\%$
$P(\begin{smallmatrix} \square & \square \\ \blacksquare & \blacksquare \end{smallmatrix}) = 0.44\%$	$P(\begin{smallmatrix} \square & \blacksquare \\ \square & \square \end{smallmatrix}) = 0.44\%$
$P(\begin{smallmatrix} \square & \blacksquare \\ \square & \square \end{smallmatrix}) = 3.11\%$	$P(\begin{smallmatrix} \square & \blacksquare \\ \blacksquare & \square \end{smallmatrix}) = 2.22\%$
$P(\begin{smallmatrix} \square & \blacksquare \\ \blacksquare & \square \end{smallmatrix}) = 4.44\%$	$P(\begin{smallmatrix} \square & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}) = 4.44\%$
$P(\blacksquare) = 0.44\%$	$P(\begin{smallmatrix} \blacksquare & \square \\ \square & \square \end{smallmatrix}) = 0.44\%$
$P(\blacksquare) = 0.00\%$	$P(\begin{smallmatrix} \blacksquare & \square \\ \blacksquare & \square \end{smallmatrix}) = 0.00\%$
$P(\begin{smallmatrix} \blacksquare & \square \\ \square & \blacksquare \end{smallmatrix}) = 0.00\%$	$P(\begin{smallmatrix} \blacksquare & \square \\ \blacksquare & \blacksquare \end{smallmatrix}) = 0.00\%$



Huffman coding

Block coding

- ▶ Hypothesis: $\frac{H(X^K)}{K}$ is converging
- ▶ This is true for example in the case of stationary process
- ▶ Average lenght of optimal code:

$$H(X^K) \leq \mathcal{L}^* < H(X^K) + 1$$

$$\frac{H(X^K)}{K} \leq \frac{\mathcal{L}^*}{K} = \mathcal{L}_S^* < \frac{H(X^K)}{K} + \frac{1}{K}$$

$$\lim_K \mathcal{L}_S^* = \lim_K \frac{H(X^K)}{K} = \mathcal{H}(X)$$

$$\mathcal{L}_S^* \rightarrow \mathcal{H}(X) \leq H(X)$$

- ▶ $\mathcal{H}(X)$ is called *entropic rate*



Huffman coding

Block coding

- ▶ Block coding:

$$\mathcal{L}_S^* \rightarrow \mathcal{H}(X) \leq H(X)$$

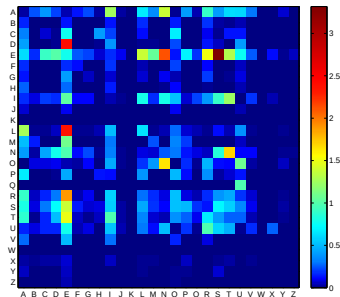
- ▶ The best performance is attained when the whole message of K symbols is encoded as a single block
- ▶ This means an alphabet of N^K symbols
- ▶ Block coding improves performance even for iid variables, since it asymptotically removes the 1-bit penalty for non-dyadic distributions



Example: Compression of French text

K=1	Letter Entropy	3.999 bpB	3.999 bpS
K=2	Digram Entropy	7.440 bpB	3.720 bpS
K=3	Trigrams Entropy	9.452 bpB	3.151 bpS

bpB: bits per block; bpS: bits per symbol, bits per letter



Digram distribution in a French text

Most common trigrams:

ait	ent	les
1.59%	1.25%	0.94%
lle	des	ant
0.78%	0.72%	0.70%
que	our	ien
0.67%	0.63%	0.60%



Limits of Huffman coding

We would like to use Huffman with larger and larger K

This is practically impossible

- ▶ Complexity is exponential with K
- ▶ The joint probability estimation for large blocks is costly and unreliable



Limits of Huffman coding

We would like to use Huffman with larger and larger K

This is practically impossible

- ▶ Complexity is exponential with K
- ▶ The joint probability estimation for large blocks is costly and unreliable

Arithmetic coding solves (at least) the first problem



Arithmetic coding

- ▶ *Arithmetic coding* allows to perform block coding or context-based coding with linear complexity
- ▶ Instead of producing the entire dictionary and then selecting the codeword, only the codeword associated to the block of n input symbols is built
- ▶ The arithmetic is suboptimal, but *asymptotically* optimal

$$\mathcal{L} \leq H(X^K) + 2$$

$$\mathcal{L}_S = \mathcal{L}/K$$

$$\lim_{K \rightarrow \infty} \mathcal{L}_S = \lim_{K \rightarrow \infty} \frac{H^K}{K} = \mathcal{H}(X)$$

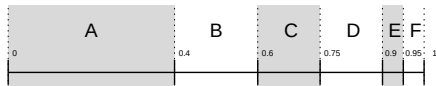
- ▶ Low-complexity encoding and decoding: two sums and two multiplications per input symbols (arithmetic operations)



Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD

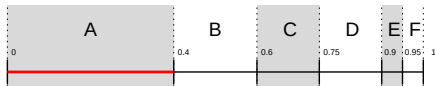




Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD

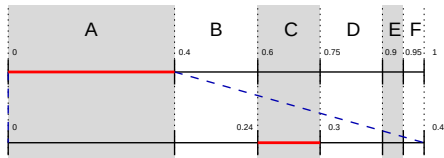




Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD

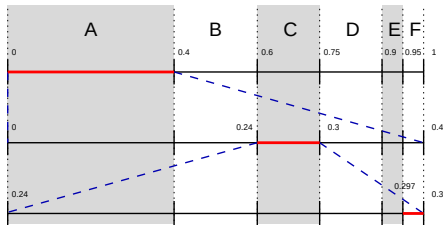




Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD

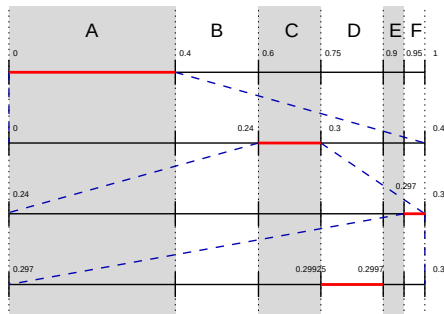




Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD

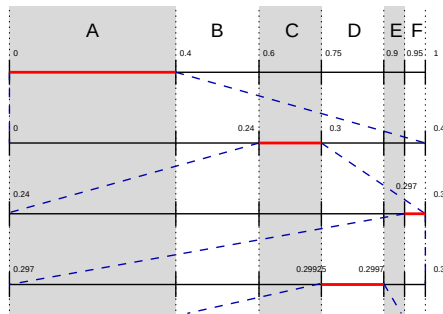




Arithmetic coding: example

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: ACFD





Arithmetic coding

- ▶ Each sequence determines an interval in $[0, 1]$
- ▶ For each new symbol, 2 multiplications and 2 sums are needed to update the interval
- ▶ A sequence is encoded as the center interval, represented with *the suitable precision*, i.e. half the size of the interval

$$\ell_i = \lceil \log_2 p_i \rceil + 1$$

$$\log_2 p_i + 1 \leq \ell_i < \log_2 p_i + 2$$

$$H(X) + 1 \leq \mathcal{L}_{AC} < H(X) + 2 \leq \mathcal{L}^* + 2$$



Arithmetic coding

- ▶ Problem: estimation of $P(X^K)$, where K can be as large as the size of the message
- ▶ In the previous exemple, symbols were supposed independent,
 $P(X^K) = \prod_{i=1}^K p(x_i)$
- ▶ Symbols statistics can be learned during the encoding process (adaptivity)



Context-based coding

- ▶ Estimation of joint probability $P(X^K)$ may be difficult
- ▶ On the other hand, often a symbol only depends on a few neighbors
- ▶ We define as *context* a set of a few, let us say N_S previous symbols that mainly impacts on the current one
- ▶ The number of possible contexts is $N_C = M^{N_S}$ at most
 - ▶ Contexts may be clustered in order to reduce complexity and improve probability estimation
- ▶ Using N_C contexts is equivalent to having N_C different arithmetic encoders and switching from the one to the other



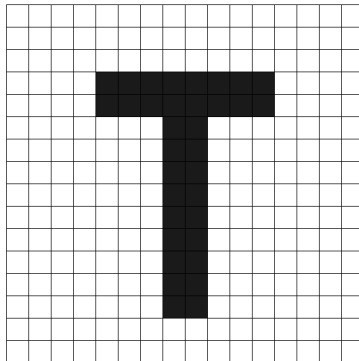
Context-based coding

$$H(X|\square) = 0.322$$

$$H(X|\blacksquare) = 0.918$$

$$H(X|Y) = 0.332P(\square) + 0.918P(\blacksquare) = 0.406$$

An arithmetic encoder can reach this rate, to be compared to $H(X, Y)/2 = 0.511$ bpp



$$P(\square|\square) = 94.1\%$$

$$P(\blacksquare|\square) = 5.9\%$$

$$P(\square|\blacksquare) = 33.3\%$$

$$P(\blacksquare|\blacksquare) = 66.7\%$$



Arithmetic coding: conclusions

Advantages

- ▶ It allows encoding long sequences of symbols with linear complexity
- ▶ Block coding: it benefits from high order dependencies and removes the penalty due to non-dyadic distributions
- ▶ Context-based coding: simple model for high order statistics
- ▶ Adaptivity: non-stationary sources



Arithmetic coding: conclusions

Disadvantages and difficulties

- ▶ Implementation may be tricky
- ▶ Need of initialization
- ▶ How to choose contexts
- ▶ Adaptivity: robust estimation demands for a potentially large training set



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques



Exponential-Golomb coding

- ▶ No initialization needed (universal coding)
- ▶ Used for encoding integers (signed or unsigned)
- ▶ No need to decide in advance the minimum and maximum value to encode: any number can be encoded
- ▶ Smaller numbers (in absolute value) get shorter codewords



Exp-Golomb coding: Unsigned Integers

- ▶ The integer $n = 0$ is encoded with the codeword "1"
- ▶ To encode the integer $n \in \{1, 2, \dots\}$, write $n + 1$ it down in binary using the minimum number of bits $b = \lfloor \log_2(n + 1) \rfloor + 1$
- ▶ Then, place $b - 1$ zeros before the obtained bitstring

n	$n + 1$ on b bits	Leading zeros	$c_U(n)$
0	1	-	1
1	10	0	010
2	11	0	011
3	100	00	00100
4	101	00	00101
5	110	00	00110
6	111	00	00111
7	1000	000	0001000
...	...	...	...



Exp-Golomb coding: Signed Integers

- ▶ First we map $n \in \mathbb{Z}$ into $m(n) \in \mathbb{N}$
 - ▶ If $n > 0$, then $m(n) = 2n - 1$
 - ▶ If $n \leq 0$, then $m(n) = -2n$
- ▶ Then we encode $m(n)$ with the unsigned Exp-Golomb code

Go to wooclap.com with the code **GYKKIN**

n	$m(n)$	$c_S(n) = c_U(m(n))$
0	0	1
1	1	010
-1	2	011
2	3	00100
-2	4	00101
3	5	00110
-3	6	00111
4	7	0001000



Dictionary-based coding

- ▶ A dictionary of most frequent sequences of symbols is built during the encoding process
- ▶ The dictionary automatically adapts to non-stationary signals
- ▶ No initialization needed (universal coding)
- ▶ Used in popular lossless coding softwares (zip, gzip, bzip, etc.)



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E
0 1

Output



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

Output

0¹



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Output



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E
0 1 0
0

Output



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 X

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0												
			0											

Output

0



Dictionary-based coding: example

Input

0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0
0

Output

0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0
0

Output

0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0												
			0											

Output

0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0											
		0	0											
			1											

Output

0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 0 X

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0											
		0	0											
			1											

Output

0 2



Dictionary-based coding: example

Input

0 0 0 **1** 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0											
		0	0											
			1											

Output

0 2



Dictionary-based coding: example

Input

0 0 0 **1 0** 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 1 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0											
		0	0											
				1										

Output

0 2



Dictionary-based coding: example

Input

0 0 0 **1 0** 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 1 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
			1											

Output

0 2



Dictionary-based coding: example

Input

0 0 0 **1 0** 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 1 ~~0~~

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
			1											

Output

0 2 1



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
0 0 0
1

Output

0 2 1



Dictionary-based coding: example

Input

0 0 0 1 **0 0** 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
			1											

Output

0 2 1



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0

New 0 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
			1											

Output

0 2 1



Dictionary-based coding: example

Input

0 0 0 1 **0 0 0** 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
			1		0									

Output

0 2 1



Dictionary-based coding: example

Input

0 0 0 1 **0 0 0** 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 0 ~~0~~

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
			1		0									

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
			1		0									

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 **0 0 0** 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1
		0	0	0
		0	1	

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 **0** 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
0 0 0
0 1

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0

Present 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
		0	1											

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0

Present 0 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
		0	1											

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1										
		0	0	0										
			0	1										

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0

Output 0 0 0 ~~0~~

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0** 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0										
		0	0	0	0										
		0	1		0										
					0										

Output

0 2 1 2 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	0								
		0	0	0	0	1								
		0	1		0									
					0									

Output

0 2 1 2 2



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 ~~X~~

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	0								
	0	0	0	0	1									
	0	1		0										
				0										

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0	0
		0	0	0	0	1
		0	1	0		
				0		

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0
1		0	0	0	0
		0	1	0	
				0	

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 0 **1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
1		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0** 1 0 0 0 0 1 0 0 0 0 0 0 1 0

Present 1 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
1		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0 1** 0 0 0 0 1 0 0 0 0 0 0 1 0

New 1 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
1		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 1 0 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1								
1		0	0	0	0	0								
		0	1		0	1								
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0

Output 1 0 X

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1								
1		0	0	0	0	0								
		0	1		0	1								
					0									

Output

0 2 1 2 2 0 4



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

New 1 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1								
1		0	0	0	0	0								
		0	1		0	1								
					0									

Output

0 2 1 2 2 0 4



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0

Output 1 0 ✕

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1	1							
1		0	0	0	0	0	0							
		0	1		0	1	0							
					0									

Output

0 2 1 2 2 0 4 4



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 0 1 0

Prefix 1 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1	1							
1		0	0	0	0	0	0							
		0	1		0	1	0							
					0									

Output

0 2 1 2 2 0 4 4



Dictionary-based coding: example

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

Prefix 1 0 0

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0	1								
1		0	0	0	0	0								
		0	1	0	0	1								
					0									

Output

0 2 1 2 2 0 4 4



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Input

0 2 1 2 2 0

New

Output



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Input

0 2 1 2 2 0

New

2 0x

Output

0x



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

x

New

2 0x

Input

0 2 1 2 2 0

Output

0x



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

x

New

3 0xy

Input

0 **2** 1 2 2 0

Output

0x

00xy



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

y

New

3 00y

Input

0 **2** 1 2 2 0

Output

000y



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0
0 0
y

New

4 1z

Input

0 2 1 2 2 0

Output

000y
0001z



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 z

1

New

4 1z

Input

0 2 1 2 2 0

Output

0001z



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
0 0 z
1

New

5 00w

Input

0 2 1 2 2 0

Output

0001z
000100w



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0
0 0 0 0
1 w

New

5 00w

Input

0 2 1 2 2 0

Output

000100w



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0
		0	0	0	0
			1		w

New

5 00w

Input

0 2 1 2 2 0

Output

000100w



Decoding: example

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1	0 0	1
	0 0	0
	0 1	

New

2 000

Input

0 2 1 2 2 0

Output

0001000



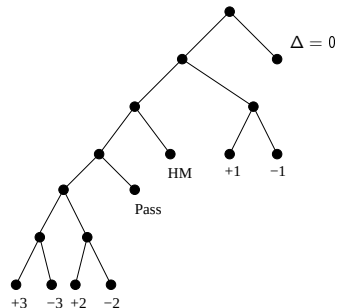
Run-Length Encoding (RLE)

- ▶ B/W image coding
- ▶ Long runs of zeros or ones
- ▶ Idea: to encode the length of the run instead of the individual pixel value



Run-Length Encoding (RLE)

- ▶ Huffman code: too large alphabet
- ▶ Huffman only on powers of two
- ▶ Each length is represented as sum of powers of two (binary representation)
- ▶ Horizontal mode: absolute length
- ▶ Vertical mode: difference with upper row (if it is ≤ 3)
- ▶ Pass: new block



Vertical mode: symbols and Huffman code



Run-Length Encoding (RLE) : Example

29	23	24
29	24	23
18	38	6 4 10
19	41	4 7 5

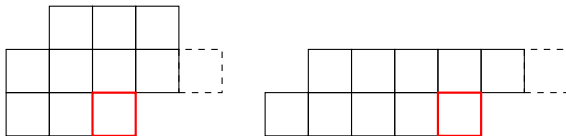
HM 16+8+4+1	16+4+2+1	16+8		
VM 0	+1	-1		
HM 16+2	32+4+2	4+2	PASS 4	PASS 8+2
VM +1	+3	-2	+3	HM 4+1



JBIG-1

Joint Bi-level Image Experts Group

- ▶ Standard ISO/IEC 11544 and recommendation ITU-T T.82
- ▶ Context-based arithmetic coding
- ▶ Progressive coding (resolution scalability)
- ▶ *Template* of 10 pixels, 2 shape with a single variable pixel position



Two rows template: faster but $\approx 5\%$ rate increase



JBIG-2

Joint Bi-level Image Experts Group

- ▶ Standard ISO/IEC 14492 and recommendation ITU-T T.88
- ▶ Image segmented into **text**, **halftones**, and **other**
- ▶ **Text**: A dictionary of symbols is created and used for coding
- ▶ **Halftone**: the gray-scale image is encoded using a dictionary of *halftone patterns*
- ▶ **Other**: Context-based arithmetic coding
- ▶ PDF files (version 1.4 and behind) can contain JBIG2 data



Quantization and coding

- ▶ Typically, quantization is followed by entropy coding (EC)
- ▶ Should we modify the desing of a quantizer knowing that it is followed by EC?
- ▶ Is it possible to estimate the performance of Q+EC?



Quantization and coding

- ▶ A generic quantizer can be modeled as a uniform quantizer (UQ) preceeded by a non-linear function $f(x)$
- ▶ We want to minimize the quantization error energy:

$$\sigma_Q^2 = \frac{\delta}{12} \int \frac{p_X(x)}{f'^2(x)} dx$$

- ▶ subject to an entropy constraint: $H(\hat{X}) \leq b$
- ▶ In the high resolution (HR) hypothesis, it can be shown that the optimum is obtained when f' is constant



Quantization and coding

In HR, it can be shown that:

- ▶ For a given MSE, the minimum quantizer entropy is obtained with UQ
- ▶ For a given quantizer entropy, the minimum MSE is obtained with UQ
- ▶ A UQ followed by an EC has a gain of 2.81 dB over a Lloyd-Max quantizer without EC in the case of Gaussian i.i.d. source
- ▶ The RD curve has still the behavior: $D \propto 2^{-2R}$